



BRAILLE DISPLAY

PRESENTED BY:

PRANAY PRATAP SINGH

RONAK AGARWALL

PIYUSH LAHOT

SATVIK GUPTA

DHEERAJ KUMAR MANDAL

BHARAT

SUMIT RAJ

UNDER THE SUPERVISION OF :

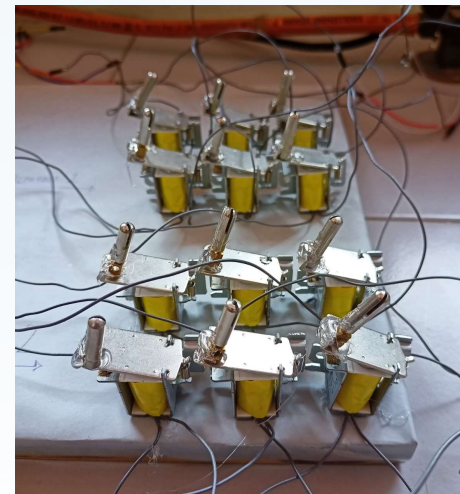
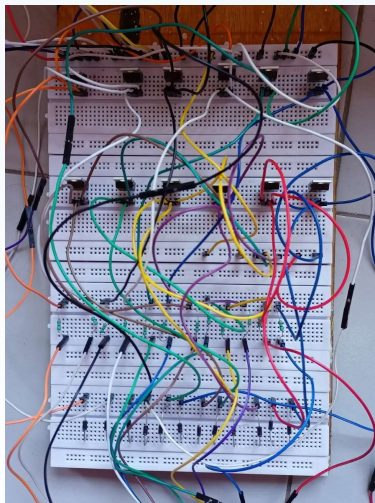
MR. MANOJ KUMAR

BHASKARACHARYA COLLEGE OF APPLIED SCIENCES

INSTRUMENTATION DEPARTMENT

INTRODUCTION

- Braille Display is a tactile device used by visually impaired individuals to read text. It converts digital information into Braille characters. Invented by Louis Braille in the 19th century. It revolutionized literacy for the blind.



A decorative graphic on the left side of the slide, consisting of several thin, dark blue lines that resemble circuit traces. These lines start from the left edge and branch out, ending in small white circles. The lines are arranged in a way that suggests a complex electronic circuit.

HOW DOES IT WORK?

Consists of a row of small, refreshable Braille cells.
Each cell has six or eight pins that can be raised or lowered.

Pins move up and down to form Braille characters.
Pins are controlled electronically.

A decorative graphic on the right side of the slide, consisting of several thin, dark blue lines that resemble circuit traces. These lines start from the right edge and branch out, ending in small white circles. The lines are arranged in a way that suggests a complex electronic circuit.

COMPONENTS OF A BRAILLE DISPLAY

1 . **Arduino Uno** : Arduino Uno is an open-source microcontroller board based on the processor ATmega328P. There are 14 digital I/O pins, 6 analog inputs, a USB connection, a power jack, an ICSP header, and a reset button. It contains all the necessary modules needed to support the microcontroller.

2. **Solenoids of rating 24 v DC supply** : Solenoid valves are control units which, when electrically energized or de-energized, either shut off or allow fluid flow. The actuator takes the form of an electromagnet.

3. **Jumper wires** : A jumper wire is an electric wire that connects remote electric circuits used for printed circuit boards.

4. **MOSFET (IRF540N)** : The IRF540N is an N-Channel MOSFET. This MOSFET can drive loads up to 23A and can support peak current up to 110A. It also has a threshold voltage of 4V, which means it can easily be driven by low voltages like 5V. Hence it is mostly used with Arduino and other microcontrollers for logic switching

5. **flyback diode (1N5819)** : The 1N5819 is a Schottky diode with a low forward voltage drop and high switching speed. It is commonly used in high frequency applications like Inverters, DC-DC converters etc.

6. **9V BATTERY**

7. **Battery Snapper Connector Clip with DC Jack**

SOFTWARE

- This code is a Python program that converts English text to Braille and sends the Braille representation to an Arduino device for further processing. It uses the PyQt5 library to create a graphical user interface (GUI) with a text input field, a "Convert" button, and an output label to display the converted Braille text. Let's go through the code step by step:
- 1. The necessary modules are imported: `sys`, `serial`, and `PyQt5` modules.
- 2. `braille\_mapping` is a dictionary that maps English characters (both uppercase and lowercase), numbers, and some punctuation symbols to their Braille representations. Each character in the English text will be replaced with its corresponding Braille representation using this mapping.
- 3. The `BrailleConverter` class is defined, which inherits from `Widget` (a base class for GUI applications in PyQt5). It represents the main window of the application.

```
import sys
import serial
from PyQt5.QtWidgets import QApplication, QWidget, QLabel,
QPlainTextEdit, QPushButton, QVBoxLayout

braille_mapping = {
    'a': '⠁', 'b': '⠃', 'c': '⠉', 'd': '⠙',
    'e': '⠑', 'f': '⠖', 'g': '⠗', 'h': '⠈',
    'i': '⠊', 'j': '⠛', 'k': '⠅', 'l': '⠍',
    'm': '⠓', 'n': '⠝', 'o': '⠥', 'p': '⠞',
    'q': '⠟', 'r': '⠡', 's': '⠎', 't': '⠥',
    'u': '⠥', 'v': '⠦', 'w': '⠡', 'x': '⠭',
    'y': '⠮', 'z': '⠵',
    'A': '⠠', 'B': '⠡', 'C': '⠠', 'D': '⠠',
    'E': '⠠', 'F': '⠠', 'G': '⠠', 'H': '⠠',
    'I': '⠠', 'J': '⠠', 'K': '⠠', 'L': '⠠',
    'M': '⠠', 'N': '⠠', 'O': '⠠', 'P': '⠠',
    'Q': '⠠', 'R': '⠠', 'S': '⠠', 'T': '⠠',
    'U': '⠠', 'V': '⠠', 'W': '⠠', 'X': '⠠',
    'Y': '⠠', 'Z': '⠠',
    '1': '⠼', '2': '⠼', '3': '⠼', '4': '⠼',
    '5': '⠼', '6': '⠼', '7': '⠼', '8': '⠼',
    '9': '⠼', '0': '⠼',
    ',': '⠂', ' ': '⠆', '!', '!', '?': '⠂', '***': '⠂',
    '***': '⠂', '***': '⠂', '***': '⠂', '***': '⠂',
    ')': '⠂', '/'': '⠂', '***': '⠂', '***': '⠂', '***': '⠂',
    '@': '⠂', '#': '⠂', '$': '⠂', '%': '⠂',
    '&': '⠂', 'A': '⠂', '***': '⠂', '***': '⠂', '***': '⠂',
    '{': '⠂', '}'': '⠂', '***': '⠂', '***': '⠂', '***': '⠂',
    '\\': '⠂', '|': '⠂', '***': '⠂', '***': '⠂',
}

class BrailleConverter(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle('English to Braille Converter')
        self.setup_ui()

        # Initialize serial communication with Arduino
        self.arduino = serial.Serial('COM3', 9600, timeout=1) # Replace 'COM3'
        with your Arduino's port

    def setup_ui(self):
        self.input_text = QPlainTextEdit()
        self.convert_button = QPushButton('Convert')
        self.output_label = QLabel()

        layout = QVBoxLayout()
        layout.addWidget(self.input_text)
        layout.addWidget(self.convert_button)
```

- 4. The ``_init_`` method of the ``BrailleConverter`` class initializes the class and sets the window title. It also calls the ``setup_ui`` method to set up the GUI elements and initializes the serial communication with the Arduino.
- 5. The ``setup_ui`` method sets up the GUI elements by creating a ``QPlainTextEdit`` widget for text input, a ``QPushButton`` widget for the conversion button, and a ``QLabel`` widget for displaying the converted Braille text. These widgets are arranged vertically using a ``QVBoxLayout``. The layout is then set as the main layout for the window.
- 6. The ``convert_text`` method is called when the conversion button is clicked. It retrieves the text from the input field, iterates over each character, and checks if it exists in the ``braille_mapping`` dictionary. If a mapping is found, the Braille representation is appended to the ``braille_text`` string. Otherwise, the original character is appended. The resulting ``braille_text`` is displayed in the output label and sent to the Arduino using serial communication.
- 7. The main part of the code checks if the program is being run directly (not imported as a module). It creates a ``QApplication`` object, which manages the GUI application, initializes an instance of the ``BrailleConverter`` class, shows the GUI window, and starts the application event loop by calling ``app.exec_()``. Overall, this code provides a simple GUI interface to convert English text to Braille and send the Braille representation to an Arduino device.

```
layout.addWidget(self.output_label)

self.setLayout(layout)

self.convert_button.clicked.connect(self.convert_text)

def convert_text(self):
    input_text = self.input_text.toPlainText()
    braille_text = ''

    for char in input_text:
        if char in braille_mapping:
            braille_text += braille_mapping[char]
        else:
            braille_text += char

    print("Braille text:", braille_text) # Add this line for debugging

    self.output_label.setText(braille_text)

    # Send braille_text to Arduino
    self.arduino.write(braille_text.encode('utf-8'))

if __name__ == '__main__':
    app = QApplication(sys.argv)
    converter = BrailleConverter()
    converter.show()
    sys.exit(app.exec_())
```

The Arduino complements the Python code . Here's an explanation of the code:

1. The `SOLENOID\_PINS` array stores the Arduino pins to which the solenoids are connected. The number of solenoids is determined by the size of the array.
2. In the `setup()` function, the solenoid pins are configured as `OUTPUT` and set to `LOW` to turn off all solenoids initially. The `Serial.begin(9600)` initializes the serial communication.
3. The `activateSolenoid()` function is defined to activate and deactivate a solenoid based on its index in the `SOLENOID\_PINS` array. It sets the corresponding pin to `HIGH`, waits for a specific duration (`delay`), and then sets the pin back to `LOW`.
4. In the `loop()` function, it checks if there is data available on the serial communication from the Python code (`Serial.available()`). If there is data, it reads a line of text until it encounters a newline character (`\n`).

```
const int SOLENOID_PINS[] = {22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44};  
// Pins connected to the solenoids  
const int NUM_SOLENOIDS = sizeof(SOLENOID_PINS) / sizeof(SOLENOID_PINS[0]);  
  
void setup()  
{  
  for (int i = 0; i < NUM_SOLENOIDS; i++)  
  {  
    pinMode(SOLENOID_PINS[i], OUTPUT);  
    digitalWrite(SOLENOID_PINS[i], LOW); // Turn off all solenoids initially  
  }  
  
  Serial.begin(9600);  
}  
  
void activateSolenoid(int index)  
{  
  digitalWrite(SOLENOID_PINS[index], HIGH); // Activate the solenoid  
  delay(3000); // Adjust the delay as needed  
  digitalWrite(SOLENOID_PINS[index], LOW); // Deactivate the solenoid  
  delay(3000); // Adjust the delay as needed  
}  
  
void loop()  
{  
  if (Serial.available())  
  {  
    String brailleText = Serial.readStringUntil('\n');  
  
    // Print brailleText to serial monitor  
    Serial.println(brailleText);  
  
    // Activate solenoids based on the Braille text received  
    for (int i = 0; i < brailleText.length(); i++)  
    {  
      char brailleChar = brailleText.charAt(i);  
      if (brailleChar == '.') // Braille dot representation for a solenoid  
      {  
        int solenoidIndex = i % NUM_SOLENOIDS; // Cycle through the solenoids  
        activateSolenoid(solenoidIndex);  
      }  
    }  
  }  
}
```




5. The received Braille text is then printed to the serial monitor using `Serial.println()`.

6. It iterates through each character in the Braille text and checks if it matches the Braille dot representation (`` . ``). If a match is found, it determines the solenoid index based on the current iteration (``i``) and activates the corresponding solenoid using the `activateSolenoid()` function.

7. The code continues to loop and listens for new Braille text from the serial communication.





BENEFITS OF BRAILLE DISPLAY

- **Access to Information** : Enables blind individuals to read digital content. Provides access to books, documents, and electronic media.
 - **Mobility** : Portable and lightweight. Can be connected to various devices like computers and smartphones.
 - **Privacy** : Allows individuals to read privately without audio output. Useful in public places like libraries or classrooms.
- 



APPLICATIONS OF BRAILLE DISPLAY

- **Education** : Facilitates learning for visually impaired students.Helps in reading textbooks, e-books, and educational materials.**Communication**:Enables blind individuals to read and respond to emails and text messages.Supports online communication platforms.**Employment** : Enhances job opportunities for visually impaired individuals.Enables access to digital documents and software interfaces.
- 

A decorative graphic on the left side of the slide consisting of several vertical and diagonal lines of varying lengths, some ending in small circles, resembling a circuit board or a stylized tree structure.

LIMITATIONS AND CHALLENGES

- **Cost** : Braille displays can be expensive, limiting accessibility for some individuals. **Technical Limitations** : Display size and number of cells can impact the amount of content that can be displayed.
 - **Learning Curve** : Requires training to learn Braille and adapt to using a Braille display effectively.
- 
- A decorative graphic on the right side of the slide, similar to the one on the left, consisting of vertical and diagonal lines with small circles at the ends.

A decorative graphic on the left side of the slide consisting of several thin, dark blue lines that branch out and end in small circles, resembling a circuit board or a stylized tree.

CONCLUSION

- Braille Display is a valuable assistive technology for visually impaired individuals. Provides access to information, enhances education, communication, and employment opportunities. Continual advancements in technology aim to make Braille displays more affordable and efficient
- 
- A decorative graphic on the right side of the slide, mirroring the one on the left, consisting of thin, dark blue lines branching out and ending in small circles.